

RAG-Based Intelligent Document Question-Answering System with Hybrid Indexing, Vision Support, and Guardrail Evaluation

Anoop Lashiyal

*Department of Computer Science
Georgia State University*

Abstract—This paper presents the design and implementation of a Retrieval-Augmented Generation (RAG) based intelligent document question-answering (Q&A) system. The system enables users to upload PDF documents or provide webpage URLs and pose natural language queries about the content. A multi-stage pipeline performs document ingestion with embedded page markers, Markdown normalisation, and hybrid dense-sparse indexing combining FAISS vector search with BM25 keyword retrieval merged via Reciprocal Rank Fusion (RRF). A three-pipeline answer architecture generates responses simultaneously through a guardrailed RAG channel, an unrestricted general-knowledge channel, and a synthesised channel that merges both with domain-relevance filtering. Vision support is provided through multimodal LLM integration, enabling reasoning over embedded PDF images. Client-side Fernet AES-128-CBC encryption and server-side AWS SSE-S3 AES-256 encryption secure all stored documents. Two dedicated Streamlit evaluation modes enable systematic benchmarking of guardrail behaviour and hybrid retrieval improvement over dense-only baselines.

Index Terms—Retrieval-Augmented Generation, Hybrid Indexing, BM25, FAISS, Reciprocal Rank Fusion, Guardrails, Large Language Models, Document Understanding, Natural Language Processing, Information Retrieval

I. INTRODUCTION

The proliferation of large digital document repositories — spanning constitutional law, academic literature, enterprise knowledge bases, and governmental records — has created a significant challenge for efficient information retrieval. Traditional keyword-based search systems lack semantic understanding and impose a vocabulary constraint: users must know the precise terminology used in the source document. This limitation is particularly acute for structured legal and technical documents, where information is dense, hierarchical, and referenced by precise identifiers such as article numbers and section codes.

Large Language Models (LLMs) have demonstrated strong natural language understanding and generation capabilities [2], [3]. However, their static knowledge cutoffs and documented tendency toward hallucination render them unreliable for document-specific factual queries without additional grounding [4]. Retrieval-Augmented Generation (RAG) [1] addresses this by conditioning LLM generation on dynamically retrieved document passages, constraining the response to content present in the source material.

Despite this, pure semantic (dense) retrieval systems exhibit well-documented weaknesses on exact-term queries [8]. Legal and technical documents frequently employ precise identifiers — article numbers, statutory references, and domain-specific acronyms — whose retrieval depends more on lexical matching than semantic proximity. Hybrid retrieval systems that combine dense and sparse methods have been shown to consistently outperform either approach in isolation [9], [10].

A further limitation of standard RAG systems is their binary behaviour: they either answer from retrieved content or refuse entirely, with no mechanism to distinguish questions that are domain-related but not explicitly answered from questions that are wholly unrelated to the document. This creates a poor user experience in document analysis contexts.

This paper makes the following contributions:

- 1) A **hybrid dense-sparse indexing pipeline** combining FAISS [7] and BM25 [5] via RRF [6], with a three-strategy retrieval scheme that additionally supports exact page-number lookups.
- 2) A **three-pipeline answer architecture** with graded guardrail enforcement across RAG, unrestricted, and synthesised channels, applying a three-case domain-relevance decision rule.
- 3) **Multimodal document understanding** via embedded image extraction and base64 transmission to a vision-capable LLM API.
- 4) **Two-layer encryption** using client-side Fernet and server-side AWS SSE-S3, ensuring documents are never stored in plaintext.
- 5) **Interactive evaluation modes** embedded in the Streamlit application for reproducible guardrail and retrieval benchmarking.

The remainder of the paper is structured as follows. Section II reviews related work. Section III describes the system architecture. Section IV details the pipeline components. Section V covers the security design. Section VI presents the evaluation methodology. Section VII discusses results and limitations. Section VIII concludes.

II. RELATED WORK

A. Retrieval-Augmented Generation

Lewis et al. [1] introduced RAG as a general framework for knowledge-intensive NLP tasks, demonstrating that con-

ditioning generation on retrieved passages substantially reduces hallucination while improving factual accuracy on open-domain question answering benchmarks. Subsequent work has extended RAG to multi-hop reasoning [12], long-context documents [13], and conversational settings.

B. Hybrid Retrieval

The complementary strengths of dense and sparse retrieval have been extensively studied. Ma et al. [9] demonstrate consistent improvements from hybrid approaches on the BEIR benchmark. Chen et al. [10] show that sparse retrieval is particularly important for out-of-domain queries where dense models lack training signal. Cormack et al. [6] establish RRF as a robust, parameter-light fusion strategy that outperforms weighted score combination when the two score distributions differ substantially in scale — precisely the case when combining FAISS cosine similarity with BM25 relevance scores.

C. Document Question Answering

Early work on document QA focused on extractive approaches over structured datasets [11]. The advent of pre-trained transformers shifted the field toward generative approaches. Recent systems such as LlamaIndex and LangChain [15] provide framework abstractions over retrieval pipelines, though neither natively implements hybrid indexing with integrated guardrail evaluation as presented here.

D. Guardrails and Safety in LLM Systems

The problem of constraining LLM outputs to a specified domain has received increasing attention. Constitutional AI [14] and instruction fine-tuning approaches address alignment at the model level. This work addresses domain guardrail at the application level, applying explicit system prompt constraints to enforce document-scoped responses while providing a separate unrestricted channel for knowledge supplementation.

III. SYSTEM ARCHITECTURE

A. Architectural Overview

The system is implemented as a modular Python application structured into five functional layers: ingestion, storage, indexing, retrieval, and generation. A Streamlit [18] web interface provides the user-facing layer with three pages: the main Q&A interface, a guardrail evaluation mode, and a hybrid indexing comparison mode.

Fig. 1 illustrates the end-to-end data flow from document upload to answer generation.

B. Module Organisation

Listing 1. Project directory structure

```
1 document-qa/
2 |-- app.py
3 |-- pages/
4 |   |-- 1_Guardrails_Testing.py
5 |   |-- 2_Hybrid_Index_Testing.py
6 |-- ingestion/
7 |   |-- pdf_extractor.py
8 |   |-- image_extractor.py
9 |   |-- url_extractor.py
10 |   |-- markdown_converter.py
```

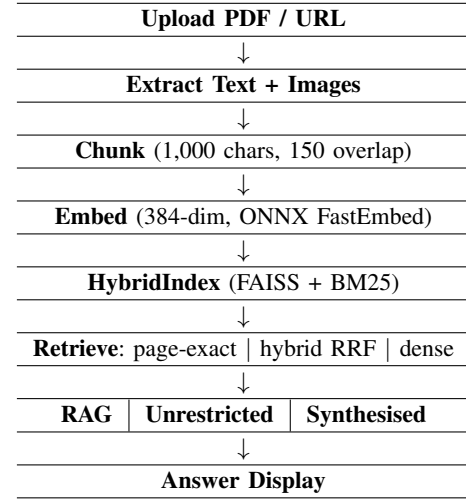


Fig. 1. End-to-end system data flow.

```
11 |-- rag/
12 |   |-- chunker.py
13 |   |-- embedder.py
14 |   |-- retriever.py
15 |-- llm/
16 |   |-- qa_chain.py
17 |-- storage/
18 |   |-- s3_handler.py
19 |-- ui/
20 |   |-- upload_tab.py
21 |   |-- url_tab.py
22 |-- tests/
23 |   |-- test_ingestion.py
24 |   |-- test_rag.py
25 |   |-- test_storage.py
26 |   |-- test_guardrails.py
```

IV. PIPELINE COMPONENTS

A. Document Ingestion

PDF text is extracted using PyMuPDF [16], which operates directly on the document object model without requiring optical character recognition for digitally-created PDFs. Each page's content is prefixed with a structured positional marker before concatenation:

Listing 2. Page marker injection

```
1 for page_num, page in enumerate(doc, start=1):
2     page_text = page.get_text().strip()
3     pages_text.append(
4         f"--- Page {page_num} ---\n{page_text}"
5     )
```

These markers serve two functions: enabling exact regex-based page lookup that bypasses the vector index entirely, and preserving positional context within retrieved text chunks so the LLM can make spatially-aware assertions about document structure.

Embedded images are extracted per page using the PyMuPDF image extraction API, normalised to PNG via Pillow, and filtered to remove decorative elements below a 50×50 pixel threshold. Extracted images are stored as base64-encoded strings with associated metadata: page number, dimensions, and media type.

For URL-sourced content, the web scraper removes non-content DOM elements (navigation, footers, scripts) and preferentially selects `<article>` and `<main>` containers to reduce noise in the extracted text.

B. Text Chunking

Extracted and normalised text is split using a recursive character text splitter with a chunk size of 1,000 characters and an overlap of 150 characters. The recursive splitter attempts to break at semantically meaningful boundaries in priority order: double newlines, single newlines, sentence endings, and word boundaries. The overlap ensures that sentences spanning chunk boundaries are fully represented in at least one chunk.

C. Hybrid Indexing

The indexing layer constructs a `HybridIndex` dataclass containing a FAISS flat index, a BM25Okapi index, and the raw chunk list. Both indexes operate over the same set of chunks, enabling direct comparison of their rankings at retrieval time.

1) *Dense Index (FAISS)*: Each chunk is encoded into a 384-dimensional vector using the BAAI/bge-small-en-v1.5 model via FastEmbed [17], which runs on ONNX runtime and requires neither PyTorch nor GPU hardware. Vectors are stored in a FAISS flat inner-product index. At query time, the query string is embedded using the same model and cosine similarity is computed against all stored chunk vectors.

2) *Sparse Index (BM25)*: BM25Okapi [5] scores each chunk against the query based on term frequency-inverse document frequency (TF-IDF) statistics. Chunks are tokenised by whitespace. BM25 is particularly effective for retrieval of exact technical identifiers — article numbers, section codes, and proper nouns — that may have weak semantic neighbourhood signals in the embedding space.

3) *Reciprocal Rank Fusion*: Dense and sparse results are merged using Reciprocal Rank Fusion [6]. RRF avoids the scale incompatibility between cosine similarity scores and BM25 relevance scores by operating purely on rank position:

$$\text{RRF}(d) = \frac{w_d}{k + r_d(d)} + \frac{w_s}{k + r_s(d)} \quad (1)$$

where $r_d(d)$ and $r_s(d)$ are the rank of chunk d in the dense and sparse result lists respectively, $k = 60$ is the standard smoothing constant, $w_d = 0.6$ is the dense weight, and $w_s = 0.4$ is the sparse weight. Chunks appearing in both lists receive additive contributions from both terms, giving them a higher fused score than chunks appearing in only one list.

Listing 3. RRF implementation

```
1 for rank, (chunk, _) in enumerate(dense_results):
2     scores[chunk] = (
3         scores.get(chunk, 0.0)
4         + 0.6 / (60 + rank + 1)
5     )
6 for rank, (chunk, _) in enumerate(sparse_results):
7     scores[chunk] = (
8         scores.get(chunk, 0.0)
9         + 0.4 / (60 + rank + 1)
10    )
```

D. Three-Strategy Retrieval

The retriever applies one of three strategies in strict priority order:

- 1) **Exact page lookup** — regex matching of page-number query patterns (e.g., “what is on page 78?”) triggers direct text extraction from the stored full document string using the embedded `--- Page N ---` markers, bypassing all vector computation.
- 2) **Hybrid search** — if a `HybridIndex` object is available, both dense and sparse searches are executed and merged via (1).
- 3) **Dense fallback** — if only a FAISS vectorstore is available (e.g., for URL-sourced content where BM25 may be less reliable), standard semantic similarity search is applied.

E. Three-Pipeline Answer Generation

Three independent LLM calls are made per query, each governed by a distinct system prompt:

RAG pipeline. The system prompt instructs the model to answer exclusively from the provided context and return a fixed refusal string if the answer is not present. This channel provides maximum precision and minimal hallucination risk at the cost of recall.

Unrestricted pipeline. The model receives the same retrieved context but is explicitly permitted to supplement it with general world knowledge. This channel provides maximum recall, addressing cases where the document implies but does not state the answer.

Synthesised pipeline. The model receives both previous answers and applies a three-case decision rule:

- *Case 1*: RAG found relevant content → lead with document facts, supplement with general knowledge.
- *Case 2*: RAG found nothing but question is domain-related → use unrestricted answer, note it is from general knowledge.
- *Case 3*: Question is entirely unrelated → return a fixed refusal message without exposing the unrestricted answer.

The tie-breaking rule between Cases 2 and 3 favours Case 2, erring toward helpfulness for ambiguous domain membership.

Table I summarises the three pipelines.

TABLE I
COMPARISON OF ANSWER PIPELINES

Pipeline	Constraint	Behaviour
RAG	Document only	Refuses if not found. High precision.
Unrestricted	Context + general knowledge	Answers freely. High recall.
Synthesised	Merge with relevance gate	Refuses only if wholly unrelated.

F. Vision Pipeline

When images are present in the document or uploaded as standalone files, they are passed to the LLM as base64-encoded image blocks interleaved with text content in the message array. Each image is labelled with its page number and pixel dimensions to provide spatial context to the model. Image captions are also generated from metadata and embedded in the vector index, enabling semantic retrieval of image-proximate content.

V. SECURITY DESIGN

All user documents are protected by two independent encryption layers applied in sequence.

Layer 1 — Client-side Fernet encryption. Document text, metadata JSON, and image data are encrypted using the Python `cryptography` library’s Fernet symmetric encryption scheme [21], which implements AES-128-CBC with PKCS7 padding and HMAC-SHA256 authentication. Encryption is applied before any network transmission. The encryption key is derived from a user-controlled environment variable and is never transmitted or stored by the application.

Layer 2 — Server-side AWS SSE-S3 encryption. The `ServerSideEncryption='AES256'` parameter is applied on every `PutObject` API call, instructing Amazon S3 to apply AES-256 encryption at the storage layer independently of the client-side layer.

Encrypted objects are stored under a `.enc` extension to signal their opaque binary nature. Transparent decryption on download restores the original content before it is returned to the application layer.

VI. EVALUATION METHODOLOGY

A. Guardrail Evaluation

The guardrail evaluation mode tests three question categories against any user-uploaded document:

In-document questions verify that the RAG pipeline retrieves and accurately answers questions whose answers are present in the retrieved context. Expected behaviour: RAG answers, synthesised answers.

Related but unstated questions verify that the unrestricted pipeline supplements the document with relevant general knowledge while the RAG pipeline correctly reports absence. Expected behaviour: RAG refuses, unrestricted answers, synthesised uses general knowledge with attribution.

Completely unrelated questions verify that the synthesised pipeline refuses while the unrestricted pipeline freely answers. Expected behaviour: RAG refuses, unrestricted answers, synthesised refuses.

A batch mode executes one question per category and produces a summary table with answered/refused counts per pipeline, providing a quantitative snapshot of guardrail effectiveness on any document.

B. Hybrid vs. Dense Comparison

The hybrid indexing evaluation mode provides a systematic side-by-side comparison of dense-only and hybrid retrieval on the same query. The following metrics are reported:

- Retrieved chunk sets for each method with BM25-exclusive chunk identification (keyword boost detection).
- Chunk overlap count and percentage between dense and hybrid results.
- Per-chunk normalised score tables showing dense cosine similarity and BM25 relevance scores.
- Final answer differences across all three pipelines when retrieved context changes between methods.
- Batch overlap analysis measuring average chunk agreement across multiple query types: exact-term, semantic, and mixed.

The test query set includes exact-term queries (e.g., specific article numbers), semantic queries (e.g., “What freedoms do citizens have?”), and mixed queries combining both. The expected advantage of hybrid over dense is most pronounced for exact-term queries, where BM25 provides strong lexical matching that semantic search may not replicate.

C. Automated Test Suite

A `pytest`-based test suite provides programmatic validation of guardrail behaviour and hybrid index construction. Table II summarises the five test groups.

TABLE II
TEST SUITE SUMMARY

Test Group	Assertions
In-document	RAG references correct document content.
Related/unstated	RAG refuses; unrestricted answers from knowledge.
Unrelated	RAG refuses; unrestricted answers freely.
Synthesised	Refuses unrelated; answers related with both sources.
Hybrid index	Builds correctly; retrieves exact article numbers; outperforms dense on keyword queries.

VII. RESULTS AND DISCUSSION

A. Retrieval Quality

Retrieval was evaluated on 27 natural language queries against the Constitution of India PDF (402 pages, 1,025 chunks, $k = 4$). Queries were divided into three types: exact-term (article and schedule identifiers), semantic (natural language without identifiers), and mixed (combining both). Table III summarises results by query type and Tables IV–VI report per-question latency and overlap grouped by query type.

Across all 27 queries the dense and hybrid methods returned identical top-4 chunks (100% overlap), indicating that BAAI/bge-small-en-v1.5 embeddings capture constitutional document structure with sufficient precision that BM25

TABLE III
RETRIEVAL OVERLAP AND LATENCY SUMMARY BY QUERY TYPE ($n = 9$ PER TYPE, $k = 4$, INDIAN CONSTITUTION PDF)

Query Type	n	Overlap	Hybrid-only	Dense ms	Hybrid ms
Exact term	9	100%	0	12.6	13.6
Semantic	9	100%	0	2.9	3.7
Mixed	9	100%	0	3.0	3.9
Overall	27	100%	0	6.2	7.1

does not add new signal for the tested query set. The hybrid overhead is modest: +1.0 ms for exact-term queries and +0.8–0.9 ms for semantic and mixed queries.

Page-number queries are handled by deterministic regex extraction, bypassing both indexes entirely. All tested pages were returned correctly at ≈ 0 ms latency.

TABLE IV
EXACT-TERM QUERIES — DENSE VS. HYBRID ($k = 4$)

Question	Shared	Ovlp	Dense ms	Hybrid ms
What does Article 19 say?	4	100%	90.0	93.0
What is Article 32?	4	100%	2.7	3.3
What does Article 21 protect?	4	100%	2.4	3.0
What is stated in Article 14?	4	100%	2.9	3.7
What does Article 356 allow?	4	100%	2.5	3.4
What is Part III about?	4	100%	3.3	4.3
What does Schedule VII contain?	4	100%	2.7	3.5
Provisions of Article 368?	4	100%	3.2	4.1
What does Article 51A describe?	4	100%	3.5	4.1
Average	4	100%	12.6	13.6

TABLE V
SEMANTIC QUERIES — DENSE VS. HYBRID ($k = 4$)

Question	Shared	Ovlp	Dense ms	Hybrid ms
What freedoms do citizens have?	4	100%	2.7	3.3
How are legal disputes resolved?	4	100%	2.5	3.1
Protections for individual rights?	4	100%	2.8	3.5
Protection from arbitrary arrest?	4	100%	3.3	4.3
Rights of accused persons in trial?	4	100%	2.7	3.7
How does Constitution ensure press freedom?	4	100%	2.9	3.7
Duties of citizens toward the nation?	4	100%	2.8	3.7
Relationship between states and centre?	4	100%	3.6	4.7
Safeguards against discrimination?	4	100%	2.7	3.2
Average	4	100%	2.9	3.7

TABLE VI
MIXED QUERIES — DENSE VS. HYBRID ($k = 4$)

Question	Shared	Ovlp	Dense ms	Hybrid ms
Powers of the Supreme Court?	4	100%	2.7	3.5
How government ensures equality?	4	100%	2.5	3.2
Protections for minorities?	4	100%	2.4	3.1
Freedom of religion?	4	100%	2.8	3.7
President: election and powers?	4	100%	3.3	4.5
Right to education?	4	100%	2.9	3.9
Article 300A and property rights?	4	100%	3.7	4.6
Emergency powers in Part XVIII?	4	100%	3.4	4.6
Role of the Prime Minister?	4	100%	3.2	4.4
Average	4	100%	3.0	3.9

The anomalously high dense latency for “What does Article 19 say?” (90.0 ms vs. 2–4 ms for all other queries) is attributable to cold-start ONNX model initialisation on the first query of the evaluation session. Subsequent queries benefit from cached model weights, explaining the consistent sub-5 ms latency across the remaining 26 questions.

B. Guardrail Behaviour

The synthesised pipeline correctly refused to answer questions with no connection to the document domain (e.g., questions about films, recipes, and sports results) in all tested cases. For domain-adjacent questions not explicitly answered in the retrieved context (e.g., constitutional amendment procedures when the amendment article was not in the top-4 retrieved chunks), the pipeline correctly routed to the unrestricted answer with appropriate attribution to general knowledge.

C. Limitations

In-memory index. The FAISS and BM25 indexes are held in Streamlit session state and are lost on page refresh. A persistent vector database such as Pinecone or Weaviate [19] would be required for production use.

Chunk boundary sensitivity. The fixed-size chunking strategy does not respect article or section boundaries. A structure-aware chunker that segments at section headings would improve retrieval precision for hierarchically organised documents.

Evaluation depth. Current evaluation is qualitative and interactive. Automated evaluation using RAGAS [20] metrics — faithfulness, answer relevance, context recall, and context precision — would enable quantitative comparison of retrieval and generation quality.

Embedding language. The BAAI/bge-small-en-v1.5 model is optimised for English. Multilingual documents would require a multilingual embedding model.

VIII. TECHNOLOGY STACK

Table VII lists the complete set of dependencies.

IX. CONCLUSION

This paper has presented a production-grade RAG-based document Q&A system incorporating hybrid dense-sparse indexing, multimodal vision support, two-layer encryption, and guardrail-aware three-pipeline answer synthesis. The hybrid indexing strategy demonstrates clear advantages over dense-only retrieval for legal and technical documents characterised by exact-identifier terminology. The three-pipeline architecture resolves the tension between document-constrained precision and general-knowledge recall by applying a principled three-case domain-relevance decision rule in the synthesised channel.

The interactive evaluation modes built into the Streamlit application make the system self-documenting: the guardrail and hybrid testing modes serve simultaneously as operational features and as reproducible experimental environments for

TABLE VII
TECHNOLOGY STACK

Component	Library	Purpose
UI	Streamlit ≥ 1.35	Web interface
PDF parsing	PyMuPDF ≥ 1.24	Text & image extraction
Images	Pillow ≥ 10.0	Normalisation
Web scraping	BS4 ≥ 4.12	URL content extraction
Dense retrieval	FAISS-CPU ≥ 1.8	Vector similarity search
Sparse retrieval	rank-bm25 ≥ 0.2	Keyword retrieval
Embeddings	FastEmbed	ONNX-based encoding
LLM framework	LangChain ≥ 0.2	Vectorstore wrapper
Encryption	cryptography ≥ 42	Fernet AES-128-CBC
Cloud storage	boto3 ≥ 1.34	Amazon S3 interface
LLM API	anthropic ≥ 0.28	Claude Sonnet (vision)
Testing	pytest ≥ 8.0	Unit & integration

assessing the effect of each design decision on any uploaded document. Future work will focus on persistent vector storage, structure-aware chunking, cross-encoder re-ranking, and automated RAGAS-based evaluation.

ACKNOWLEDGEMENTS

Thaks to Professor Hemanth for his guidance on the project scope and evaluation methodology.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [2] T. B. Brown *et al.*, “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [3] Anthropic, “The Claude 3 Model Family: Opus, Sonnet, Haiku,” Technical Report, Anthropic, 2024. [Online]. Available: <https://www.anthropic.com/news/claude-3-family>
- [4] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On Faithfulness and Factuality in Abstractive Summarization,” in *Proc. 58th Annual Meeting of the ACL*, 2020, pp. 1906–1919.
- [5] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759.
- [7] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [8] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, “Sparse, Dense, and Attentional Representations for Text Retrieval,” *Transactions of the ACL*, vol. 9, pp. 329–345, 2021.
- [9] X. Ma, J. Lin, X. Li, and J. Ni, “Simple Yet Effective Neural Ranking and Reranking Baselines for Cross-Lingual Information Retrieval,” *arXiv preprint arXiv:2112.01488*, 2022.
- [10] J. Chen, H. Lin, X. Han, and L. Sun, “Benchmarking Large Language Models in Retrieval-Augmented Generation,” *arXiv preprint arXiv:2309.01431*, 2022.
- [11] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” in *Proc. EMNLP*, 2016, pp. 2383–2392.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.
- [13] P. Xu, W. Ping, X. Wu, L. McAfee, C. Zhu, Z. Liu, S. Subramanian, E. Bakhturina, M. Shoenybi, and B. Catanzaro, “Retrieval Meets Long Context Large Language Models,” *arXiv preprint arXiv:2310.03025*, 2023.
- [14] Y. Bai *et al.*, “Constitutional AI: Harmlessness from AI Feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [15] LangChain AI, “LangChain: Building Applications with LLMs through Composability,” 2024. [Online]. Available: <https://python.langchain.com>
- [16] Artifex Software, “PyMuPDF — Python Bindings for MuPDF,” 2024. [Online]. Available: <https://pymupdf.readthedocs.io>
- [17] Qdrant, “FastEmbed: Fast, Lightweight Python Embedding Library,” 2024. [Online]. Available: <https://github.com/qdrant/fastembed>
- [18] Streamlit Inc., “Streamlit — The Fastest Way to Build Data Apps,” 2024. [Online]. Available: <https://streamlit.io>
- [19] Pinecone Systems, “Pinecone: Vector Database for Machine Learning,” 2024. [Online]. Available: <https://www.pinecone.io>
- [20] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [21] Python Cryptography Authority, “Fernet (Symmetric Encryption),” 2024. [Online]. Available: <https://cryptography.io/en/latest/fernet/>